Practical 1: MLOps Project Template Builder

Aim: To create a reusable folder structure for an ML project.

Step 1: Create the folder

Open PowerShell:

```
cd C:\Users\ACER\mlops_practical
mkdir Practical1
cd Practical1
```

Step 2: Create a Python file

```
notepad template_builder.py
```

Paste this code into Notepad and save it:

```
from pathlib import Pathproject_name = "my_ml_project"folders = [
"data/raw",    "data/processed",    "notebooks",    "src",    "models",
"reports"]files = {    "README.md": "# My ML Project\n\nA reusable MLOps
project template.",    "requirements.txt": "pandas\nscikit-
learn\njoblib\n",    "src/train.py": "# Model training code goes here\n",
"src/predict.py": "# Prediction code goes here\n",
"reports/metrics.txt": "Model metrics will be stored here.\n"}root =
Path(project_name)for folder in folders:    (root /
folder).mkdir(parents=True, exist_ok=True)for filename, content in
files.items():    path = root / filename    path.parent.mkdir(parents=True,
exist_ok=True)    if not path.exists():        path.write_text(content,
encoding="utf-8")print("MLOps project template created
successfully!")print(f"Project location: {root.resolve()}")
```

Step 3: Run the program

```
python template_builder.py
```

Expected output:

```
MLOps project template created successfully!
Project location: ...\Practical1\my_ml_project
```

Step 4: Verify the structure

```
tree my_ml_project /F
```

You should see folders such as `data`, `src`, `models`, `reports`, and files such as `README.md` and `requirements.txt`.

Result: A reusable MLOps project template was created successfully.

Practical 4: Interpretable AI — Enhancing Model Transparency

Aim: To train a machine-learning model and understand which features influence its predictions.

We'll use a Random Forest classifier and its feature-importance scores.

Step 1: Create the folder

```
cd C:\Users\ACER\mlops_practical
mkdir Practical4
cd Practical4
```

Step 2: Install the libraries

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install scikit-learn pandas matplotlib
```

If the activation policy blocks the command, run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
```

Step 3: Create the Python file

```
notepad interpretable_ai.py
```

Paste and save this code:

```
import pandas as pdimport matplotlib.pyplot as pltfrom sklearn.datasets
import load_breast_cancerfrom sklearn.ensemble import
RandomForestClassifierfrom sklearn.model_selection import
train_test_splitfrom sklearn.metrics import accuracy_score,
classification_report# Load sample datasetdata = load_breast_cancer()X =
pd.DataFrame(data.data, columns=data.feature_names)y = data.target# Split
data into training and testing setsX_train, X_test, y_train, y_test =
train_test_split(    X, y, test_size=0.2, random_state=42, stratify=y)#
Train the modelmodel = RandomForestClassifier(n_estimators=100,
random_state=42)model.fit(X_train, y_train)# Evaluate the modelpredictions
= model.predict(X_test)print("Model Accuracy:",
round(accuracy_score(y_test, predictions), 4))print("\nClassification
Report:")print(classification_report(y_test, predictions,
target_names=data.target_names))# Explain the model using feature
importanceimportance = pd.Series(    model.feature_importances_,
index=X.columns).sort_values(ascending=False)print("\nTop 10 Important
Features:")
```

Step 4: Run the program

```
python interpretable_ai.py
```

Expected results:

- Model accuracy is displayed.
- A classification report is printed.
- The top 10 important features are listed.
- A chart appears showing the features' importance.
- `feature_importance.png` is saved in the folder.

If the chart window opens, close it to return to PowerShell.

Result: A Random Forest model was trained, evaluated, and interpreted using feature importance.

Viva: Interpretable AI helps us understand why a model may make a prediction. Feature importance indicates which inputs contributed most to the Random Forest's predictions, but it does not prove causation.

Practical 6: End-to-End ML Pipeline Orchestration with MLflow

Aim: To train a machine-learning model and track its parameters, performance metrics, and artifacts using MLflow.

Step 1: Create the folder

```
cd C:\Users\ACER\mlops_practical
mkdir Practical6
cd Practical6
```

Step 2: Create and activate a virtual environment

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Install the required libraries:

```
python -m pip install --upgrade pip
pip install mlflow scikit-learn
```

Step 3: Create the MLflow program

```
notepad mlflow_pipeline.py
```

Paste and save this code:

```
import mlflowimport mlflow.sklearnfrom sklearn.datasets import
load_irisfrom sklearn.model_selection import train_test_splitfrom
sklearn.ensemble import RandomForestClassifierfrom sklearn.metrics import
accuracy_score# Load datasetiris = load_iris()X, y = iris.data,
iris.target# Split datasetX_train, X_test, y_train, y_test =
train_test_split(    X, y, test_size=0.2, random_state=42, stratify=y)# Set
experiment namemlflow.set_experiment("Iris_Classification")# Start tracking
the experimentwith mlflow.start_run():    n_estimators = 100    model =
RandomForestClassifier(        n_estimators=n_estimators,
random_state=42    )    model.fit(X_train, y_train)    predictions =
model.predict(X_test)    accuracy = accuracy_score(y_test, predictions)
# Log parameters and metrics    mlflow.log_param("model_type",
"RandomForest")
```

Step 4: Run the pipeline

```
python mlflow_pipeline.py
```

Expected output includes a successful training message and an accuracy value. MLflow should also create a local `mlruns` tracking directory.

Step 5: Open the MLflow dashboard

In the same folder and virtual environment, run:

```
mlflow ui --host 127.0.0.1 --port 5000
```

Keep this terminal open. Open your browser and visit:

Open MLflow dashboard

You should be able to view the `Iris_Classification` experiment and its recorded run, including parameters and accuracy.

To stop the dashboard, press Ctrl + C in the terminal running `mlflow ui`.

Result: An ML model was trained, and its parameters, accuracy, and model artifact were tracked using MLflow.

Viva questions:

- What is MLflow? A tool for managing and tracking machine-learning experiments and models.
- What is a parameter? A model setting, such as the number of trees.
- What is a metric? A measurement of model performance, such as accuracy.
- What is experiment tracking? Recording runs so that different model experiments can be compared.

Practical 8: Big Data Exploration with Dask for Scalable Computing

Aim: To explore and analyze a dataset using Dask, which supports parallel and lazy computations on data.

Step 1: Create the folder

```
cd C:\Users\ACER\mlops_practical
mkdir Practical8
cd Practical8
```

Step 2: Create and activate the environment

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Install Dask and Pandas:

```
python -m pip install --upgrade pip
pip install "dask[dataframe]" pandas
```

Step 3: Create the Python program

```
notepad dask_analysis.py
```

Paste and save this code:

```
import pandas as pdimport dask.dataframe as dd# Create a sample datasetdata
= {    "student": ["A", "B", "C", "D", "E", "F"],    "department": ["AIDS",
"CSE", "AIDS", "CSE", "AIDS", "CSE"],    "marks": [85, 72, 90, 65, 78, 88],
"attendance": [90, 80, 95, 70, 85,
92]}pd.DataFrame(data).to_csv("students.csv", index=False)print("Dataset
created successfully.")# Load CSV using Daskdf =
dd.read_csv("students.csv")print("\nDataset preview:")print(df.head())#
Perform computationsprint("\nTotal number of
records:")print(df.shape[0].compute())print("\nAverage
marks:")print(df["marks"].mean().compute())print("\nAverage marks by
department:")print(df.groupby("department")["marks"].mean().compute())print
("\nStudents with marks above 80:")print(df[df["marks"] >
80].compute())print("\nDask data analysis completed successfully.")
```

Step 4: Run the program

```
python dask_analysis.py
```

Expected results:

- A file named `students.csv` is created.
- The dataset preview is displayed.

- Total records: `6`.
- Average marks: `79.6667` approximately.
- Average marks by department and students scoring above 80 are displayed.